

Doctor & Staff Module — Deep Dive

Presenter: **Ma Noo Noo Aung** Package: `com.hospitalqueue.api` Companion to: Queue System Team Split

Everything needed to explain and defend this module: what each endpoint does, how a queue moves through its states, the exact code the doctor's "Call Next" button and staff's emergency confirm/reject buttons run, and a rehearsed demo script with anticipated questions.

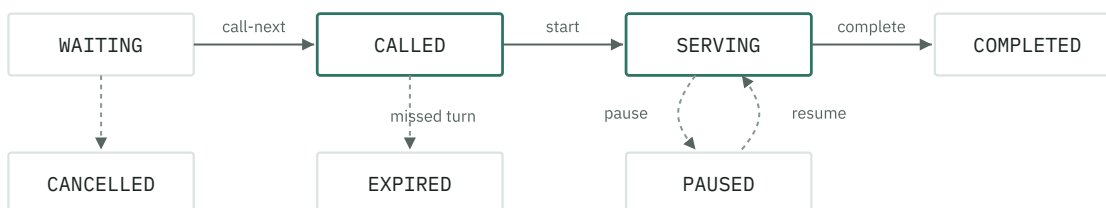
```
api/DoctorApiController.java    api/StaffApiController.java    service/QueueService.java
rule/EmergencyRule.java    routes/doctor.queue.tsx    routes/staff.emergency.tsx
```

Where this module sits

The **Smart Queue Engine** (Member 2 — `rule/QueueRule`, `DoctorAvailabilityRule`, `EmergencyRule`) decides queue *order*. This module is the two portals that actually **act** on that order: the doctor pulling patients out of it one at a time, and staff sitting at the front desk keeping emergency cases honest before they jump the line. Neither portal touches the database directly — both call `QueueService`, which is the single place queue state actually changes.

Queue state machine `Queue.java`

Every queue entry is a row moving left to right through these states. Doctor actions drive the top row; **PAUSED** lets a doctor step away without losing the patient's place; the bottom row are the ways a queue entry can end without being served.



Doctor API 7 endpoints

METHOD	PATH	WHAT IT DOES
GET	/api/doctor/{id}/dashboard	Full dashboard: called/serving/waiting queues, today's appointments, stats — all patients batch-loaded in one query.
GET	/api/doctor/{id}/queue	Lighter, polled every 8s by the live queue screen — same shape, fewer fields.
POST	/api/doctor/{id}/availability	Toggle or explicitly set available/unavailable (goes off duty / on break).
POST	/api/doctor/{id}/queue/call-next	Pulls the highest-priority WAITING patient and marks them CALLED. Refuses if someone is already CALLED or SERVING.
POST	/api/doctor/{id}/queue/start	CALLED → SERVING for whoever is currently called.
POST	/api/doctor/{id}/queue/complete	SERVING → COMPLETED, then recalculates wait times for everyone still waiting.
POST	/api/doctor/{id}/queue/pause · /resume	SERVING ⇌ PAUSED, without losing the patient's place.
GET	/api/doctor/{id}/appointments · /history · /profile	Read-only: this doctor's appointments, past queue history, profile card.

Staff API 9 endpoints

METHOD	PATH	WHAT IT DOES
GET	/api/staff/dashboard	Front-desk stats: today's totals, emergency-pending count, EMERGENCY/APPOINTMENT/NORMAL breakdown, active vs. on-break doctors.
GET	/api/staff/emergencies	All emergency-flagged queues awaiting staff triage confirmation.
POST	/api/staff/emergencies/{id}/confirm	Marks the case genuine — emergencyConfirmed = true, patient notified they've moved to the front.

POST	/api/staff/emergencies/{id}/reject	Cancels the queue entry with reason "Emergency rejected by staff".
GET	/api/staff/queue-monitor	Live per-doctor board: now-serving ticket, waiting count, completed-today count — filterable by department/doctor.
GET	/api/staff/queues/reassignable	Every WAITING queue, for moving a patient to another doctor.
POST	/api/staff/queues/{id}/reassign	Moves one queue entry to a different doctor.
POST	/api/staff/doctors/{id}/unavailable	Force a doctor off duty from the staff side (e.g. called away).
GET	/api/staff/appointments · /reports · /profile	Hospital-wide appointment list, department reports, staff's own profile card.

Flagship flow: Call Next Patient

This is the one action most worth walking through line by line — it is where the Doctor module and the Smart Queue Engine (Member 2) actually meet.

```
SERVICE/QUEUESERVICE.JAVA — CALLNEXT(DOCTORID)

// 1. Refuse if a patient is already CALLED or SERVING
if (currentConsultation(doctorId) != null)
    throw new IllegalStateException("Finish or start the current consultation first.");

// 2. Pull every WAITING patient for this doctor
List<Queue> waiting = queueRepository.findWaitingQueuesByDoctor(doctorId);
if (waiting.isEmpty()) return null;

// 3. EmergencyRule picks the front of the line: EMERGENCY > APPOINTMENT > NORMAL,
//    then by original position (FIFO within the same priority)
Queue next = emergencyRule.reorderByPriority(waiting).get(0);
queueRepository.callNext(next.getQueueId());    // WAITING -> CALLED
notificationService.notify(next.getPatientId(), "It's your turn! ...");

// 4. Recompute estimated wait for everyone still behind, in one batched UPDATE
updateRemainingWaitTimes(doctorId);
```

```
RULE/EMERGENCYRULE.JAVA — THE ACTUAL SORT

waitingQueues.sort((q1, q2) -> {
    int p1 = priorityRank(q1), p2 = priorityRank(q2);    // EMERGENCY=1, APPOINTMENT=2, NORMAL=3
    if (p1 != p2) return Integer.compare(p1, p2);
```

```
return Integer.compare(q1.getPosition(), q2.getPosition()); // tie-break: FIFO
});
```

Good question to be ready for: "Does an emergency jump the queue before staff confirms it?" — Yes. `priorityRank()` sorts on the queue's `priority` field alone, which is set to `EMERGENCY` the moment the patient (or staff) flags it — not on `emergencyConfirmed`. Staff confirmation doesn't gate ordering; it's the front desk verifying the case is genuine and formally logging who handled it, while the system has already fast-tracked it for safety. If asked "what if it's a false alarm," the answer is `reject`: the entry is cancelled outright rather than merely demoted.

Emergency confirm / reject flow

Confirm

`StaffApiController.confirmEmergency` → `QueueService.confirmEmergency(queueId)` sets `emergencyConfirmed = true` and notifies the patient they've been moved to the front — logged, not reordered again (it was already first by priority).

Reject

`StaffApiController.rejectEmergency` calls `queueRepository.cancel(queueId, "Emergency rejected by staff")` directly — the entry becomes `CANCELLED`, not demoted to `NORMAL`.

Frontend behaviour worth knowing

`doctor.queue.tsx`

Polls `getDoctorDashboard` every **8 seconds** via `setInterval`. The "Call Next" button disables itself whenever `data.serving || data.called` is truthy — the same one-active-consultation rule the backend enforces, just mirrored in the UI so the button never fires a request the API would reject anyway.

`staff.emergency.tsx`

Splits cases into pending / resolved tabs client-side from one fetched list, plus a `critical` vs. `fastTrack` split by priority for the red "Act Now" banner. Confirm/reject optimistically patch local state instead of refetching the whole list.

Live demo script ~4 min each

- 1 Open Doctor Portal → Consultation Queue**
Point out the waiting list is already ordered — explain it's `EmergencyRule`, not arrival time.
- 2 Click "Call Next Patient"**

Ticket moves to the console as CALLED; note the button is now disabled — can't call two at once.

3 **Click "Start Consultation" → "Complete"**

Show the waiting list's estimated-wait minutes shift for everyone else after Complete.

4 **Switch to Staff Portal → Emergency Cases**

Show a pending case in the "Pending" tab with the red Critical Cases counter.

5 **Click "Confirm" on a case**

Explain: it was already first in line by priority — confirming logs staff sign-off, it doesn't re-sort anything.

Likely questions

Why can't a doctor call a second patient while one is already called?

`callNext()` checks `currentConsultation()` first, which is CALLED-or-SERVING — one active consultation per doctor at a time, enforced server-side (not just the disabled button).

What happens to wait-time estimates when a patient completes?

`completeCurrent()` calls `updateRemainingWaitTimes()`, which recomputes every remaining WAITING patient's estimate as `position × avg consultation minutes` (plus a lunch-break offset) and writes them all in one batched UPDATE.

Can staff move a patient to a different doctor?

Yes — `/api/staff/queues/reassignable` lists candidates, `/api/staff/queues/{id}/reassign` moves one queue entry to a different `doctorId` via `QueueService.reassignDoctor`.

How does PAUSED differ from just not calling the next patient?

PAUSED is specifically for a patient already SERVING — the doctor steps away mid-consultation without marking it COMPLETED. `resume()` puts them back to SERVING; the queue slot isn't given up.

Companion to *Queue System Team Split*. Package boundaries: `api.DoctorApiController`, `api.StaffApiController`, `service.QueueService`, `rule.EmergencyRule` — the code these slides describe is exactly what ships.

Appendix — full source code 6 files

The complete, unedited files behind every endpoint and flow above — for reading straight from during questions.

src/main/java/com/hospitalqueue/api/DoctorApiController.java

349 lines · Doctor API

```
1 package com.hospitalqueue.api;
2
3 import com.hospitalqueue.model.Appointment;
4 import com.hospitalqueue.model.Department;
5 import com.hospitalqueue.model.Doctor;
6 import com.hospitalqueue.model.Patient;
7 import com.hospitalqueue.model.Queue;
8 import com.hospitalqueue.repository.AppointmentRepository;
9 import com.hospitalqueue.repository.DepartmentRepository;
10 import com.hospitalqueue.repository.DoctorRepository;
11 import com.hospitalqueue.repository.PatientRepository;
12 import com.hospitalqueue.repository.QueueRepository;
13 import com.hospitalqueue.service.QueueService;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.web.bind.annotation.*;
17
18 import java.util.*;
19 import java.util.stream.Collectors;
20
21 @RestController
22 @RequestMapping("/api/doctor")
23 public class DoctorApiController {
24
25     private final QueueService queueService;
26     private final QueueRepository queueRepository;
27     private final DoctorRepository doctorRepository;
28     private final PatientRepository patientRepository;
29     private final DepartmentRepository departmentRepository;
30     private final AppointmentRepository appointmentRepository;
31
32     public DoctorApiController(QueueService queueService,
33                               QueueRepository queueRepository,
34                               DoctorRepository doctorRepository,
35                               PatientRepository patientRepository,
36                               DepartmentRepository departmentRepository,
37                               AppointmentRepository appointmentRepository) {
38         this.queueService = queueService;
39         this.queueRepository = queueRepository;
40         this.doctorRepository = doctorRepository;
41         this.patientRepository = patientRepository;
42         this.departmentRepository = departmentRepository;
43         this.appointmentRepository = appointmentRepository;
44     }
45
46     @GetMapping("/{doctorId}/dashboard")
47     public ResponseEntity<?> getDashboard(@PathVariable String doctorId) {
48         Doctor doctor = doctorRepository.findById(doctorId);
49         if (doctor == null) {
50             return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", "Doctor not found"));
51         }
52
53         Department dept = departmentRepository.findById(doctor.getDepartmentId());
54         String deptName = dept != null ? dept.getDepartmentName() : "General Medicine";
55
56         List<Queue> activeQueues = queueRepository.findActiveQueuesForDoctor(doctor.getDoctorId());
```

```

57     Queue called = activeQueues.stream().filter(q ->
"CALLED".equals(q.getStatus()))
    .findFirst().orElse(null);
58     Queue serving = activeQueues.stream().filter(q ->
"SERVING".equals(q.getStatus()))
    .findFirst().orElse(null);
59     List<Queue> waiting = activeQueues.stream().filter(q -> "WAITING".equals(q.getStatus()))
60         .collect(Collectors.toList());
61
62     List<Appointment> todayAppointments = appointmentRepository.findTodayByDoctor(doctor.getDoctorId());
63     int completedToday = (int) queueRepository.countCompletedTodayForDoctor(doctor.getDoctorId());
64
65     Set<String> patientIds = new HashSet<>();
66     for (Queue q : activeQueues) {
67         if (q.getPatientId() != null)
68             patientIds.add(q.getPatientId());
69     }
70     for (Appointment a : todayAppointments) {
71         if (a.getPatientId() != null)
72             patientIds.add(a.getPatientId());
73     }
74
75     Map<String, Patient> ptMap = patientIds.isEmpty()
76         ? Collections.emptyMap()
77         : patientRepository.findByIds(patientIds).stream()
78             .collect(Collectors.toMap(p -> p.getPatientId(), p -> p, (a, b) -> a));
79
80     Map<String, Object> stats = new LinkedHashMap<>();
81     stats.put("totalPatientsToday",
82         waiting.size() + (called != null ? 1 : 0) + (serving != null ? 1 : 0) + completedToday);
83     stats.put("waitingCount", waiting.size());
84     stats.put("completedCount", completedToday);
85     stats.put("averageConsultationMinutes",
86         doctor.getAverageConsultationMinutes() > 0 ? doctor.getAverageConsultationMinutes() : 15);
87
88     Map<String, Object> doctorInfo = new LinkedHashMap<>();
89     doctorInfo.put("id", doctor.getDoctorId());
90     doctorInfo.put("name", doctor.getName());
91     doctorInfo.put("doctorCode", doctor.getDoctorCode());
92     doctorInfo.put("department", deptName);
93     doctorInfo.put("departmentId", doctor.getDepartmentId());
94     doctorInfo.put("specialization",
95         doctor.getSpecialization() != null ? doctor.getSpecialization() : "Consultant");
96     doctorInfo.put("available", doctor.isAvailable());
97     doctorInfo.put("phone", doctor.getPhone());
98     doctorInfo.put("email", doctor.getEmail());
99     doctorInfo.put("maxQueueSize", doctor.getMaxQueueSize());
100    doctorInfo.put("averageConsultationMinutes", doctor.getAverageConsultationMinutes());
101
102    Map<String, Object> res = new LinkedHashMap<>();
103    res.put("doctor", doctorInfo);
104    res.put("stats", stats);
105    res.put("called", called != null ? mapQueueWithPatients(called, ptMap, doctor, dept) : null);
106    res.put("serving", serving != null ? mapQueueWithPatients(serving, ptMap, doctor, dept) : null);
107    res.put("waiting", waiting.stream().map(q -> mapQueueWithPatients(q, ptMap, doctor,
dept)).collect(Collectors.toList()));
108    res.put("todayAppointments", todayAppointments.stream().map(a -> mapAppointmentWithPatients(a, ptMap,
deptName))
109        .collect(Collectors.toList()));
110
111    return ResponseEntity.ok(res);
112 }
113
114 @GetMapping("/{doctorId}/queue")
115 public ResponseEntity<?> getQueueState(@PathVariable String doctorId) {
116     Doctor doctor = doctorRepository.findById(doctorId);
117     if (doctor == null) {
118         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", "Doctor not found"));

```

```

119     }
120
121     Queue called = queueService.called(doctorId);
122     Queue serving = queueService.serving(doctorId);
123     List<Queue> waiting = queueService.waitingForDoctor(doctorId);
124
125     // All entries belong to this one doctor: resolve the doctor's department
126     // once and batch-load every patient in one query, instead of the 3
127     // separate DB round trips (patient/doctor/department) mapQueue() used to
128     // make per queue row – this endpoint is polled frequently by the live
129     // queue screen.
130     Department dept = departmentRepository.findById(doctor.getDepartmentId());
131     Set<String> patientIds = new HashSet<>();
132     if (called != null) patientIds.add(called.getPatientId());
133     if (serving != null) patientIds.add(serving.getPatientId());
134     for (Queue q : waiting) {
135         if (q.getPatientId() != null) patientIds.add(q.getPatientId());
136     }
137     Map<String, Patient> ptMap = patientIds.isEmpty()
138         ? Collections.emptyMap()
139         : patientRepository.findByIds(patientIds).stream()
140             .collect(Collectors.toMap(Patient::getPatientId, p -> p, (a, b) -> a));
141
142     Map<String, Object> res = new LinkedHashMap<>();
143     res.put("doctor", Map.of(
144         "id", doctor.getDoctorId(),
145         "name", doctor.getName(),
146         "doctorCode", doctor.getDoctorCode(),
147         "available", doctor.isAvailable()));
148     res.put("called", called != null ? mapQueueWithPatients(called, ptMap, doctor, dept) : null);
149     res.put("serving", serving != null ? mapQueueWithPatients(serving, ptMap, doctor, dept) : null);
150     res.put("waiting", waiting.stream().map(q -> mapQueueWithPatients(q, ptMap, doctor,
dept)).collect(Collectors.toList()));
151
152     return ResponseEntity.ok(res);
153 }
154
155 @PostMapping("/{doctorId}/availability")
156 public ResponseEntity<?> toggleAvailability(@PathVariable String doctorId,
157     @RequestBody(required = false) Map<String, Object> payload) {
158     Doctor doctor = doctorRepository.findById(doctorId);
159     if (doctor == null) {
160         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", "Doctor not found"));
161     }
162
163     boolean newAvailability;
164     if (payload != null && payload.containsKey("available")) {
165         newAvailability = Boolean.parseBoolean(payload.get("available").toString());
166     } else {
167         newAvailability = !doctor.isAvailable();
168     }
169
170     doctorRepository.setAvailability(doctorId, newAvailability);
171     return ResponseEntity.ok(Map.of(
172         "success", true,
173         "available", newAvailability,
174         "message", "Availability updated to " + (newAvailability ? "AVAILABLE" : "UNAVAILABLE")));
175 }
176
177 @PostMapping("/{doctorId}/queue/call-next")
178 public ResponseEntity<?> callNext(@PathVariable String doctorId) {
179     if (queueService.currentConsultation(doctorId) != null) {
180         return ResponseEntity.badRequest().body(Map.of("error",
181             "A patient is already called or in consultation. Complete or start the current consultation
first."));
182     }

```

```

183     Queue next = queueService.callNext(doctorId);
184     if (next == null) {
185         return ResponseEntity.ok(Map.of("message", "No patients waiting in queue.", "called", null));
186     }
187     return ResponseEntity.ok(Map.of("message", "Patient called successfully.", "called", mapQueue(next)));
188 }
189
190 @PostMapping("/{doctorId}/queue/start")
191 public ResponseEntity<?> startConsultation(@PathVariable String doctorId) {
192     boolean started = queueService.startConsultation(doctorId);
193     if (!started) {
194         return ResponseEntity.badRequest().body(Map.of("success", false, "error", "No called patient to start
consultation with."));
195     }
196     return ResponseEntity.ok(Map.of("success", true, "message", "Consultation started."));
197 }
198
199 @PostMapping("/{doctorId}/queue/complete")
200 public ResponseEntity<?> completeConsultation(@PathVariable String doctorId) {
201     boolean completed = queueService.completeCurrent(doctorId);
202     if (!completed) {
203         return ResponseEntity.badRequest().body(Map.of("success", false, "error", "No active consultation to
complete."));
204     }
205     return ResponseEntity.ok(Map.of("success", true, "message", "Consultation completed."));
206 }
207
208 @PostMapping("/{doctorId}/queue/pause")
209 public ResponseEntity<?> pauseConsultation(@PathVariable String doctorId) {
210     boolean paused = queueService.pauseCurrent(doctorId);
211     if (!paused) {
212         return ResponseEntity.badRequest().body(Map.of("success", false, "error", "Failed to pause."));
213     }
214     return ResponseEntity.ok(Map.of("success", true, "message", "Consultation paused."));
215 }
216
217 @PostMapping("/{doctorId}/queue/resume")
218 public ResponseEntity<?> resumeConsultation(@PathVariable String doctorId) {
219     boolean resumed = queueService.resumeCurrent(doctorId);
220     if (!resumed) {
221         return ResponseEntity.badRequest().body(Map.of("success", false, "error", "Failed to resume."));
222     }
223     return ResponseEntity.ok(Map.of("success", true, "message", "Consultation resumed."));
224 }
225
226 @GetMapping("/{doctorId}/appointments")
227 public List<Map<String, Object>> getAppointments(@PathVariable String doctorId) {
228     List<Appointment> list = appointmentRepository.findByDoctor(doctorId);
229
230     Doctor doctor = doctorRepository.findById(doctorId);
231     Department dept = doctor != null ? departmentRepository.findById(doctor.getDepartmentId()) : null;
232     String deptName = dept != null ? dept.getDepartmentName() : null;
233
234     Set<String> patientIds = list.stream().map(Appointment::getPatientId)
235         .filter(Objects::nonNull).collect(Collectors.toSet());
236     Map<String, Patient> ptMap = patientIds.isEmpty()
237         ? Collections.emptyMap()
238         : patientRepository.findByIds(patientIds).stream()
239             .collect(Collectors.toMap(Patient::getPatientId, p -> p, (a, b) -> a));
240
241     return list.stream().map(a -> mapAppointmentWithPatients(a, ptMap,
deptName)).collect(Collectors.toList());
242 }
243
244 @GetMapping("/{doctorId}/history")
245 public List<Map<String, Object>> getHistory(@PathVariable String doctorId) {

```

```

246         return queueRepository.findHistoryForDoctor(doctorId).stream().map(h -> {
247             Map<String, Object> map = new LinkedHashMap<>();
248             map.put("id", h.getHistoryId());
249             map.put("queueNumber", h.getQueueNumber());
250             map.put("patientId", h.getPatientId());
251             map.put("status", h.getStatus());
252             map.put("reason", h.getChangeReason());
253             map.put("createdAt", h.getCreatedAt() != null ? h.getCreatedAt().toString() : "");
254             return map;
255         }).collect(Collectors.toList());
256     }
257
258     @GetMapping("/{doctorId}/profile")
259     public ResponseEntity<> getProfile(@PathVariable String doctorId) {
260         Doctor doc = doctorRepository.findById(doctorId);
261         if (doc == null) {
262             return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error", "Doctor not found"));
263         }
264         Department dept = departmentRepository.findById(doc.getDepartmentId());
265         Map<String, Object> res = new LinkedHashMap<>();
266         res.put("id", doc.getDoctorId());
267         res.put("name", doc.getName());
268         res.put("doctorCode", doc.getDoctorCode());
269         res.put("department", dept != null ? dept.getDepartmentName() : "General");
270         res.put("departmentId", doc.getDepartmentId());
271         res.put("specialization", doc.getSpecialization() != null ? doc.getSpecialization() : "General
Physician");
272         res.put("phone", doc.getPhone() != null ? doc.getPhone() : "09 12345678");
273         res.put("email", doc.getEmail() != null ? doc.getEmail() : "doctor@hospital.com");
274         res.put("available", doc.isAvailable());
275         res.put("averageConsultationMinutes", doc.getAverageConsultationMinutes());
276         res.put("maxQueueSize", doc.getMaxQueueSize());
277         res.put("queueOpenTime", doc.getQueueOpenTime() != null ? doc.getQueueOpenTime().toString() : "09:00");
278         res.put("queueCloseTime", doc.getQueueCloseTime() != null ? doc.getQueueCloseTime().toString() :
"17:00");
279         return ResponseEntity.ok(res);
280     }
281
282     /**
283      * Shared queue -> DTO mapping. All lookups (patient/doctor/department) are
284      * passed in already-resolved rather than fetched here, so callers dealing
285      * with a whole list (all for the same doctor) fetch the doctor/department
286      * once and the patients in one batch instead of per queue row.
287      */
288     private Map<String, Object> mapQueueCore(Queue q, Patient pt, Doctor doc, Department dept) {
289         Map<String, Object> map = new LinkedHashMap<>();
290         map.put("id", q.getQueueId());
291         map.put("queueNumber", q.getQueueNumber());
292         map.put("patientId", q.getPatientId());
293         map.put("patientName", pt != null ? pt.getName() : "Patient");
294         map.put("doctorId", q.getDoctorId());
295         map.put("doctorName", doc != null ? doc.getName() : "Doctor");
296         map.put("departmentId", doc != null ? doc.getDepartmentId() : q.getDepartmentId());
297         map.put("departmentName", dept != null ? dept.getDepartmentName() : "General Medicine");
298         map.put("position", q.getPosition());
299         map.put("status", q.getStatus());
300         map.put("priority", q.getPriority());
301         map.put("type", q.isEmergency() ? "EMERGENCY"
: ("APPOINTMENT".equalsIgnoreCase(q.getPriority()) ? "APPOINTMENT" : "NORMAL"));
302         map.put("emergency", q.isEmergency());
303         map.put("estimatedWaitingMinutes", q.getEstimatedWaitingTime());
304         map.put("waitingMinutes", q.getEstimatedWaitingTime());
305         map.put("createdAt", q.getCreatedAt() != null ? q.getCreatedAt().toString() : "");
306         return map;
307     }
308 }
309

```

```

310  /** Single-item convenience overload (e.g. mapping the one queue entry just called). */
311  private Map<String, Object> mapQueue(Queue q) {
312      Patient pt = patientRepository.findById(q.getPatientId());
313      Doctor doc = doctorRepository.findById(q.getDoctorId());
314      Department dept = doc != null ? departmentRepository.findById(doc.getDepartmentId()) : null;
315      return mapQueueCore(q, pt, doc, dept);
316  }
317
318  private Map<String, Object> mapQueueWithPatients(Queue q, Map<String, Patient> ptMap) {
319      Patient pt = ptMap != null ? ptMap.get(q.getPatientId()) : patientRepository.findById(q.getPatientId());
320      Doctor doc = doctorRepository.findById(q.getDoctorId());
321      Department dept = doc != null ? departmentRepository.findById(doc.getDepartmentId()) : null;
322      return mapQueueCore(q, pt, doc, dept);
323  }
324
325  /**
326   * Batched overload for lists of queue entries that all belong to the same
327   * doctor (every doctor-facing endpoint here) – the doctor and department are
328   * resolved once by the caller instead of once per queue row.
329   */
330  private Map<String, Object> mapQueueWithPatients(Queue q, Map<String, Patient> ptMap, Doctor doc, Department
dept) {
331      Patient pt = ptMap != null ? ptMap.get(q.getPatientId()) : null;
332      return mapQueueCore(q, pt, doc, dept);
333  }
334
335  private Map<String, Object> mapAppointmentWithPatients(Appointment a, Map<String, Patient> ptMap, String
deptName) {
336      Patient pt = ptMap != null ? ptMap.get(a.getPatientId()) : patientRepository.findById(a.getPatientId());
337      Map<String, Object> map = new LinkedHashMap<>();
338      map.put("id", a.getAppointmentId());
339      map.put("patientId", a.getPatientId());
340      map.put("patientName", pt != null ? pt.getName() : "Patient");
341      map.put("doctorId", a.getDoctorId());
342      map.put("departmentName", deptName != null ? deptName : "General");
343      map.put("appointmentDate", a.getAppointmentDate() != null ? a.getAppointmentDate().toString() : "");
344      map.put("appointmentTime", a.getAppointmentTime() != null ? a.getAppointmentTime().toString() : "");
345      map.put("status", a.getStatus());
346      map.put("notes", a.getNotes());
347      return map;
348  }
349  }

```

src/main/java/com/hospitalqueue/api/StaffApiController.java

374 lines · Staff API

```

1  package com.hospitalqueue.api;
2
3  import com.hospitalqueue.model.Appointment;
4  import com.hospitalqueue.model.Department;
5  import com.hospitalqueue.model.Doctor;
6  import com.hospitalqueue.model.Patient;
7  import com.hospitalqueue.model.Queue;
8  import com.hospitalqueue.model.Staff;
9  import com.hospitalqueue.repository.AppointmentRepository;
10 import com.hospitalqueue.repository.DepartmentRepository;
11 import com.hospitalqueue.repository.DoctorRepository;
12 import com.hospitalqueue.repository.PatientRepository;
13 import com.hospitalqueue.repository.QueueRepository;
14 import com.hospitalqueue.repository.StaffRepository;
15 import com.hospitalqueue.service.QueueService;
16 import org.springframework.http.ResponseEntity;
17 import org.springframework.web.bind.annotation.*;
18

```

```

19 import java.time.LocalDate;
20 import java.util.Collections;
21 import java.util.LinkedHashMap;
22 import java.util.List;
23 import java.util.Map;
24 import java.util.stream.Collectors;
25
26 @RestController
27 @RequestMapping("/api/staff")
28 public class StaffApiController {
29
30     private final QueueRepository queueRepository;
31     private final QueueService queueService;
32     private final DoctorRepository doctorRepository;
33     private final DepartmentRepository departmentRepository;
34     private final PatientRepository patientRepository;
35     private final AppointmentRepository appointmentRepository;
36     private final StaffRepository staffRepository;
37
38     public StaffApiController(QueueRepository queueRepository,
39                             QueueService queueService,
40                             DoctorRepository doctorRepository,
41                             DepartmentRepository departmentRepository,
42                             PatientRepository patientRepository,
43                             AppointmentRepository appointmentRepository,
44                             StaffRepository staffRepository) {
45         this.queueRepository = queueRepository;
46         this.queueService = queueService;
47         this.doctorRepository = doctorRepository;
48         this.departmentRepository = departmentRepository;
49         this.patientRepository = patientRepository;
50         this.appointmentRepository = appointmentRepository;
51         this.staffRepository = staffRepository;
52     }
53
54     @GetMapping("/dashboard")
55     public Map<String, Object> getDashboard() {
56         Map<String, Long> stats = queueRepository.getDashboardStatistics();
57         long totalToday = stats.getOrDefault("totalToday", 0L);
58         long emergencyPending = stats.getOrDefault("emergencyPending", 0L);
59         long waiting = stats.getOrDefault("waiting", 0L);
60         long completed = stats.getOrDefault("completed", 0L);
61         long emergCount = stats.getOrDefault("emergToday", 0L);
62         long apptCount = stats.getOrDefault("apptToday", 0L);
63         long normCount = stats.getOrDefault("normToday", 0L);
64
65         List<Doctor> doctors = doctorRepository.findAll();
66         long activeDoctors = doctors.stream().filter(d -> d.isAvailable()).count();
67         long onBreakDoctors = doctors.size() - activeDoctors;
68
69         long sum = Math.max(1, emergCount + apptCount + normCount);
70
71         Map<String, Object> emergItem = new LinkedHashMap<>();
72         emergItem.put("label", "Emergency");
73         emergItem.put("value", emergCount);
74         emergItem.put("percent", (int) Math.round((emergCount * 100.0) / sum));
75         emergItem.put("color", "var(--color-status-emergency)");
76
77         Map<String, Object> apptItem = new LinkedHashMap<>();
78         apptItem.put("label", "Appointment");
79         apptItem.put("value", apptCount);
80         apptItem.put("percent", (int) Math.round((apptCount * 100.0) / sum));
81         apptItem.put("color", "var(--color-status-appointment)");
82
83         Map<String, Object> normItem = new LinkedHashMap<>();
84         normItem.put("label", "Normal");

```

```

85     normItem.put("value", normCount);
86     normItem.put("percent", (int) Math.round((normCount * 100.0) / sum));
87     normItem.put("color", "var(--color-status-normal)");
88
89     Map<String, Object> h1 = new LinkedHashMap<>();
90     h1.put("label", "Queues Completed");
91     h1.put("value", completed);
92
93     Map<String, Object> h2 = new LinkedHashMap<>();
94     h2.put("label", "Patients Waiting");
95     h2.put("value", waiting);
96
97     double avgWait = queueRepository.getAverageWaitingTimeToday();
98
99     Map<String, Object> h3 = new LinkedHashMap<>();
100    h3.put("label", "Average Waiting Time");
101    h3.put("value", (int) Math.round(avgWait));
102
103    Map<String, Object> h4 = new LinkedHashMap<>();
104    h4.put("label", "Doctor On Break");
105    h4.put("value", onBreakDoctors);
106
107    Map<String, Object> data = new LinkedHashMap<>();
108    data.put("totalQueuesToday", totalToday);
109    data.put("emergencyPending", emergencyPending);
110    data.put("activeDoctors", activeDoctors);
111    data.put("checkedInAppointments", apptCount);
112    data.put("breakdown", List.of(emergItem, apptItem, normItem));
113    data.put("highlights", List.of(h1, h2, h3, h4));
114
115    return data;
116 }
117
118 @GetMapping("/reports")
119 public Map<String, Object> getReports(@RequestParam(required = false) String type,
120     @RequestParam(required = false) String date) {
121     LocalDate targetDate = null;
122     if (date != null && !date.isBlank()) {
123         try {
124             targetDate = LocalDate.parse(date);
125         } catch (Exception ignored) {}
126     }
127
128     if (targetDate == null) {
129         targetDate = LocalDate.now();
130     }
131
132     List<Map<String, Object>> rows = queueRepository.getDepartmentReports(targetDate);
133
134     long totalQueues = rows.stream().mapToLong(r -> ((Number) r.get("totalQueues")).longValue()).sum();
135     long completed = rows.stream().mapToLong(r -> ((Number) r.get("completed")).longValue()).sum();
136     long cancelled = rows.stream().mapToLong(r -> ((Number) r.get("cancelled")).longValue()).sum();
137     long missed = rows.stream().mapToLong(r -> ((Number) r.get("missed")).longValue()).sum();
138
139     return Map.of(
140         "date", targetDate.toString(),
141         "totalQueues", totalQueues,
142         "completed", completed,
143         "cancelled", cancelled,
144         "missed", missed,
145         "rows", rows);
146 }
147
148 @GetMapping("/profile")
149 public Map<String, Object> getProfile(@RequestParam(required = false) String staffId) {
150     Staff staff = null;

```

```

151         if (staffId != null) {
152             staff = staffRepository.findById(staffId);
153         }
154         if (staff == null) {
155             List<Staff> list = staffRepository.findAll();
156             if (!list.isEmpty())
157                 staff = list.get(0);
158         }
159
160         Map<String, Object> map = new LinkedHashMap<>();
161         map.put("id", staff != null ? staff.getStaffId() : "STF001");
162         map.put("staffCode", staff != null ? staff.getStaffCode() : "STF001");
163         map.put("name", staff != null ? staff.getName() : "Staff Member");
164         map.put("role", staff != null ? staff.getRole() : "Receptionist");
165         map.put("department", "Reception / Triage");
166         map.put("phone", staff != null ? staff.getPhone() : "09 987654321");
167         map.put("email", staff != null && staff.getEmail() != null ? staff.getEmail() : "staff@hospital.com");
168         map.put("workingShift", "8:00 AM - 4:00 PM");
169         map.put("joiningDate", "12 Mar 2023");
170         map.put("avatarUrl", "");
171         return map;
172     }
173
174     @GetMapping("/emergencies")
175     public List<Map<String, Object>> getEmergencies() {
176         List<Queue> emergencies = queueRepository.findPendingEmergencies();
177         if (emergencies.isEmpty())
178             return Collections.emptyList();
179
180         Map<String, Patient> ptMap = patientRepository.findAll().stream()
181             .collect(Collectors.toMap(p -> p.getPatientId(), p -> p, (a, b) -> a));
182
183         return emergencies.stream().map(q -> {
184             Patient pt = ptMap.get(q.getPatientId());
185             Map<String, Object> map = new LinkedHashMap<>();
186             map.put("id", q.getQueueId());
187             map.put("patientCode",
188                 pt != null ? pt.getPatientId() : (q.getPatientId() != null ? q.getPatientId() : "P001"));
189             map.put("patientId", q.getPatientId());
190             map.put("patientName", pt != null ? pt.getName() : "Emergency Patient");
191             map.put("symptoms", "Emergency Case - " + q.getQueueNumber());
192             map.put("requestedTime", q.getCreatedAt() != null ? q.getCreatedAt().toLocalTime().toString() :
"10:15 AM");
193             map.put("priority", "HIGH");
194             map.put("resolved", q.isEmergencyConfirmed());
195             map.put("handledBy", q.isEmergencyConfirmed() ? "Staff on Duty" : null);
196             map.put("confirmedTime", q.isEmergencyConfirmed() ? "Confirmed" : null);
197             return map;
198         }).collect(Collectors.toList());
199     }
200
201     @PostMapping("/emergencies/{queueId}/confirm")
202     public ResponseEntity<?> confirmEmergency(@PathVariable long queueId) {
203         queueService.confirmEmergency(queueId);
204         return ResponseEntity.ok(Map.of(
205             "id", queueId,
206             "resolved", true,
207             "handledBy", "Staff",
208             "confirmedTime", "Just now"));
209     }
210
211     @PostMapping("/emergencies/{queueId}/reject")
212     public ResponseEntity<?> rejectEmergency(@PathVariable long queueId) {
213         queueRepository.cancel(queueId, "Emergency rejected by staff");
214         return ResponseEntity.ok(Map.of("success", true));
215     }

```

```

216
217 @GetMapping("/queue-monitor")
218 public List<Map<String, Object>> getQueueMonitor(@RequestParam(required = false) String departmentName,
219 @RequestParam(required = false) String doctorName) {
220     List<Doctor> doctors = queueService.enrichAvailability(doctorRepository.findAll());
221     Map<Integer, Department> deptMap = departmentRepository.findAll().stream()
222         .collect(Collectors.toMap(d -> d.getDepartmentId(), d -> d, (a, b) -> a));
223     Map<String, Queue> servingMap = queueRepository.findServingByAllDoctors();
224     Map<String, Integer> waitingMap = queueRepository.countWaitingByAllDoctors();
225     Map<String, Long> completedMap = queueRepository.countCompletedTodayByAllDoctors();
226
227     return doctors.stream().map(doc -> {
228         Department dept = deptMap.get(doc.getDepartmentId());
229         String deptName = dept != null ? dept.getDepartmentName() : "General";
230         Queue serving = servingMap.get(doc.getDoctorId());
231         int waitingCount = waitingMap.getOrDefault(doc.getDoctorId(), 0);
232         long completedCount = completedMap.getOrDefault(doc.getDoctorId(), 0L);
233
234         Map<String, Object> row = new LinkedHashMap<>();
235         row.put("departmentName", deptName);
236         row.put("doctorName", doc.getName());
237         row.put("status", doc.isAvailable() ? "CONSULTING" : "UNAVAILABLE");
238         row.put("nowServing", serving != null ? serving.getQueueNumber() : null);
239         row.put("waiting", waitingCount);
240         row.put("completed", (int) completedCount);
241         row.put("avgWaitingMinutes", (int) doc.getAverageConsultationMinutes());
242         return row;
243     }).filter(r -> (departmentName == null || "ALL".equalsIgnoreCase(departmentName)
244         || departmentName.equalsIgnoreCase((String) r.get("departmentName")))
245         && (doctorName == null || "ALL".equalsIgnoreCase(doctorName)
246         || doctorName.equalsIgnoreCase((String) r.get("doctorName"))))
247         .collect(Collectors.toList());
248 }
249
250 @GetMapping("/queues/reassignable")
251 public List<Map<String, Object>> getReassignableQueues(@RequestParam(required = false) String departmentName)
252 {
253     List<Queue> waiting = queueRepository.findAllWaiting();
254     if (waiting.isEmpty())
255         return Collections.emptyList();
256
257     if (departmentName != null && !departmentName.isBlank() && !"ALL".equalsIgnoreCase(departmentName)) {
258         Department targetDept = departmentRepository.findByName(departmentName);
259         if (targetDept != null) {
260             int targetDeptId = targetDept.getDepartmentId();
261             waiting = waiting.stream().filter(q -> q.getDepartmentId() == targetDeptId)
262                 .collect(Collectors.toList());
263         }
264     }
265
266     Map<String, Doctor> docMap = doctorRepository.findAll().stream()
267         .collect(Collectors.toMap(d -> d.getDoctorId(), d -> d, (a, b) -> a));
268     Map<Integer, Department> deptMap = departmentRepository.findAll().stream()
269         .collect(Collectors.toMap(d -> d.getDepartmentId(), d -> d, (a, b) -> a));
270     Map<String, Patient> ptMap = patientRepository.findAll().stream()
271         .collect(Collectors.toMap(p -> p.getPatientId(), p -> p, (a, b) -> a));
272
273     return waiting.stream().map(q -> {
274         Doctor doc = docMap.get(q.getDoctorId());
275         Department dept = deptMap.get(q.getDepartmentId());
276         Patient pt = ptMap.get(q.getPatientId());
277
278         Map<String, Object> map = new LinkedHashMap<>();
279         map.put("id", q.getQueueId());
280         map.put("queueNumber", q.getQueueNumber());
281         map.put("patientId", q.getPatientId());

```

```

281         map.put("patientName", pt != null ? pt.getName() : "Patient");
282         map.put("departmentId", q.getDepartmentId());
283         map.put("departmentName", dept != null ? dept.getDepartmentName() : "General");
284         map.put("doctorId", q.getDoctorId());
285         map.put("doctorName", doc != null ? doc.getName() : "Dr. Specialist");
286         map.put("position", q.getPosition());
287         map.put("estimatedWaitingMinutes", q.getEstimatedWaitingTime());
288         map.put("waitingMinutes", q.getEstimatedWaitingTime());
289         map.put("status", q.getStatus());
290         map.put("type", q.isEmergency() ? "EMERGENCY"
291             : ("APPOINTMENT".equalsIgnoreCase(q.getPriority()) ? "APPOINTMENT" : "NORMAL"));
292         map.put("createdAt", q.getCreatedAt() != null ? q.getCreatedAt().toString() : "");
293         return map;
294     }).collect(Collectors.toList());
295 }
296
297 public static class ReassignDto {
298     public String doctorId;
299     public String reason;
300 }
301
302 @PostMapping("/queues/{queueId}/reassign")
303 public ResponseEntity<> reassignQueue(@PathVariable long queueId, @RequestBody ReassignDto req) {
304     if (req == null || req.doctorId == null) {
305         return ResponseEntity.badRequest().body(Map.of("error", "Target doctor ID is required.));
306     }
307     queueService.reassignDoctor(queueId, req.doctorId);
308     return ResponseEntity.ok(Map.of("success", true));
309 }
310
311 public static class DoctorUnavailableDto {
312     public String reason;
313 }
314
315 @PostMapping("/doctors/{doctorId}/unavailable")
316 public ResponseEntity<> setDoctorUnavailable(@PathVariable String doctorId,
317     @RequestBody(required = false) DoctorUnavailableDto req) {
318     Doctor doc = doctorRepository.findById(doctorId);
319     if (doc != null) {
320         doctorRepository.setAvailability(doctorId, false);
321     }
322     return ResponseEntity.ok(Map.of("success", true));
323 }
324
325 @GetMapping("/appointments")
326 public List<Map<String, Object>> getAllAppointments() {
327     List<Appointment> list = appointmentRepository.findAll();
328     if (list.isEmpty())
329         return Collections.emptyList();
330
331     Map<String, Doctor> docMap = doctorRepository.findAll().stream()
332         .collect(Collectors.toMap(d -> d.getDoctorId(), d -> d, (a, b) -> a));
333     Map<Integer, Department> deptMap = departmentRepository.findAll().stream()
334         .collect(Collectors.toMap(d -> d.getDepartmentId(), d -> d, (a, b) -> a));
335     Map<String, Patient> ptMap = patientRepository.findAll().stream()
336         .collect(Collectors.toMap(p -> p.getPatientId(), p -> p, (a, b) -> a));
337
338     return list.stream().map(a -> {
339         Doctor doc = docMap.get(a.getDoctorId());
340         Department dept = deptMap.get(a.getDepartmentId());
341         Patient pt = ptMap.get(a.getPatientId());
342         Map<String, Object> map = new LinkedHashMap<>();
343         map.put("id", a.getId());
344         map.put("patientId", a.getPatientId());
345         map.put("patientName", pt != null ? pt.getName() : "Patient");
346         map.put("doctorId", a.getDoctorId());

```

```

347         map.put("doctorName", doc != null ? doc.getName() : "Dr. Specialist");
348         map.put("departmentName", dept != null ? dept.getDepartmentName() : "General");
349         map.put("appointmentDate", a.getAppointmentDate() != null ? a.getAppointmentDate().toString() : "");
350         map.put("appointmentTime", a.getAppointmentTime() != null ? a.getAppointmentTime().toString() : "");
351         map.put("status", a.getStatus());
352         return map;
353     }).collect(Collectors.toList());
354 }
355
356 @GetMapping("/notifications")
357 public List<Map<String, Object>> getNotifications() {
358     return List.of(
359         Map.of("id", 1, "title", "Emergency case added", "message",
360             "Emergency patient waiting for triage confirmation.", "time", "10:15 AM", "read", false,
361             "important", true, "category", "EMERGENCY"),
362         Map.of("id", 2, "title", "Queue reassigned", "message", "Queue reassigned successfully.", "time",
363             "10:08 AM", "read", false, "important", false, "category", "QUEUE"),
364         Map.of("id", 3, "title", "Doctor on break", "message", "Dr. Khin Khin (Pediatrics) is now on
break.",
365             "time", "09:50 AM", "read", false, "important", false, "category", "DOCTOR"),
366         Map.of("id", 4, "title", "Schedule updated", "message", "Doctor schedules have been updated.",
"time",
367             "Yesterday", "read", true, "important", true, "category", "SCHEDULE"));
368     }
369
370 @PostMapping("/notifications/read-all")
371 public ResponseEntity<?> markAllNotificationsRead() {
372     return ResponseEntity.ok(Map.of("success", true));
373 }
374 }

```

src/main/java/com/hospitalqueue/service/QueueService.java

402 lines · Smart Queue Engine service (shared with Member 2)

```

1  package com.hospitalqueue.service;
2
3  import com.hospitalqueue.model.Appointment;
4  import com.hospitalqueue.model.Department;
5  import com.hospitalqueue.model.Doctor;
6  import com.hospitalqueue.model.Queue;
7  import com.hospitalqueue.ml.WaitTimePredictionService;
8  import com.hospitalqueue.repository.AppointmentRepository;
9  import com.hospitalqueue.repository.DepartmentRepository;
10 import com.hospitalqueue.repository.DoctorRepository;
11 import com.hospitalqueue.repository.PatientRepository;
12 import com.hospitalqueue.repository.QueueRepository;
13 import com.hospitalqueue.repository.SymptomRepository;
14 import com.hospitalqueue.rule.DoctorAvailabilityRule;
15 import com.hospitalqueue.rule.EmergencyRule;
16 import com.hospitalqueue.rule.QueueRule;
17 import com.hospitalqueue.util.QueueNumberGenerator;
18 import org.slf4j.Logger;
19 import org.slf4j.LoggerFactory;
20 import org.springframework.stereotype.Service;
21
22 import java.time.LocalDateTime;
23 import java.time.LocalTime;
24 import java.util.List;
25 import java.util.Map;
26
27 /**
28  * Smart Queue Engine service - the core business logic of the system.
29  */

```

```

30 @Service
31 public class QueueService {
32
33     private static final Logger log = LoggerFactory.getLogger(QueueService.class);
34
35     private final QueueRepository queueRepository;
36     private final AppointmentRepository appointmentRepository;
37     private final DoctorRepository doctorRepository;
38     private final DepartmentRepository departmentRepository;
39     private final PatientRepository patientRepository;
40     private final WaitingTimeService waitingTimeService;
41     private final NotificationService notificationService;
42     private final QueueRule queueRule;
43     private final EmergencyRule emergencyRule;
44     private final DoctorAvailabilityRule doctorAvailabilityRule;
45     private final WaitTimePredictionService waitTimePredictionService;
46     private final SymptomRepository symptomRepository;
47
48     public QueueService(QueueRepository queueRepository,
49         AppointmentRepository appointmentRepository,
50         DoctorRepository doctorRepository,
51         DepartmentRepository departmentRepository,
52         PatientRepository patientRepository,
53         WaitingTimeService waitingTimeService,
54         NotificationService notificationService,
55         QueueRule queueRule,
56         EmergencyRule emergencyRule,
57         DoctorAvailabilityRule doctorAvailabilityRule,
58         WaitTimePredictionService waitTimePredictionService,
59         SymptomRepository symptomRepository) {
60         this.queueRepository = queueRepository;
61         this.appointmentRepository = appointmentRepository;
62         this.doctorRepository = doctorRepository;
63         this.departmentRepository = departmentRepository;
64         this.patientRepository = patientRepository;
65         this.waitingTimeService = waitingTimeService;
66         this.notificationService = notificationService;
67         this.queueRule = queueRule;
68         this.emergencyRule = emergencyRule;
69         this.doctorAvailabilityRule = doctorAvailabilityRule;
70         this.waitTimePredictionService = waitTimePredictionService;
71         this.symptomRepository = symptomRepository;
72     }
73
74     /**
75      * Doctor availability detection: true when the doctor can currently accept new
76      * patients.
77      */
78     public boolean isDoctorAvailable(String doctorId) {
79         Doctor doctor = doctorRepository.findById(doctorId);
80         if (doctor == null) {
81             return false;
82         }
83         int waitingCount = queueRepository.countWaitingByDoctor(doctorId);
84         return doctorAvailabilityRule.isAvailable(doctor, waitingCount, LocalTime.now());
85     }
86
87     /**
88      * Marks a list of doctors with their computed availability and sorts available
89      * first.
90      */
91     public List<Doctor> enrichAvailability(List<Doctor> doctors) {
92         Map<String, Integer> waitingCounts = queueRepository.countWaitingByAllDoctors();
93         return doctorAvailabilityRule.markAndSortAvailable(doctors, waitingCounts, LocalTime.now());
94     }
95

```

```

96     public Queue createQueue(String patientId, String doctorId, int departmentId, String priority, boolean
emergency,
97         String source) {
98         return createQueue(patientId, doctorId, departmentId, priority, emergency, source,
99             java.util.Collections.emptyList());
100     }
101
102     /**
103     * Creates a new queue for a patient applying Rule 1 (one active queue),
104     * Rule 5 (department validation), Rule 8 (registration window) and
105     * Rule 11 (capacity). Generates the queue number and waiting time.
106     */
107     public synchronized Queue createQueue(String patientId, String doctorId, int departmentId, String priority,
boolean emergency,
108         String source, List<Integer> symptomIds) {
109         com.hospitalqueue.model.Patient patient = patientRepository.findById(patientId);
110         if (patient == null) {
111             throw new IllegalStateException("Patient not found. Please log in again.");
112         }
113         // Use the canonical patient_id (findById tolerates lookup by phone/email/etc.)
114         patientId = patient.getPatientId();
115
116         Queue activeQueue = queueRepository.findActiveQueueByPatientId(patientId);
117         if (queueRule.hasActiveQueue(activeQueue)) {
118             throw new IllegalStateException("Rule 1: you already have an active queue. Cancel it first.");
119         }
120
121         Doctor doctor = doctorRepository.findById(doctorId);
122         if (doctor == null) {
123             throw new IllegalStateException("Doctor not found. Please choose a doctor.");
124         }
125         if (!doctor.isAvailable()) {
126             throw new IllegalStateException("This doctor is currently unavailable. Please choose another
doctor.");
127         }
128         if (doctor.getDepartmentId() != departmentId) {
129             throw new IllegalStateException("Rule 5: the selected doctor does not belong to the chosen
department.");
130         }
131
132         LocalTime open = doctor.getQueueOpenTime() != null ? doctor.getQueueOpenTime() : LocalTime.of(9, 0);
133         LocalTime close = doctor.getQueueCloseTime() != null ? doctor.getQueueCloseTime() : LocalTime.of(16, 30);
134         LocalTime now = LocalTime.now();
135
136         // Enforce hospital operating hours (09:00 AM - 4:30 PM) for non-emergencies
137         if (!emergency) {
138             if (now.isBefore(open) || now.isAfter(close)) {
139                 throw new IllegalStateException("Hospital queues are accepted between " + open + " and " + close
140                     + ". Please register during operating hours or select Emergency if urgent.");
141             }
142         }
143
144         // Calculate dynamic session capacity based on actual session duration and
145         // average consultation time (using historical data when available)
146         long avgConsultMinutes = waitingTimeService.getDoctorAverageConsultationMinutes(doctor);
147         long sessionMinutes = java.time.Duration.between(open, close).toMinutes();
148         if (sessionMinutes <= 0) sessionMinutes = 450; // 7.5 hours default
149
150         // Subtract the 1-hour lunch break from effective session time
151         long effectiveSessionMinutes = Math.max(60, sessionMinutes - 60);
152
153         // Calculate max capacity: effective session time / average consultation time
154         long calculatedMaxCapacity = Math.max(5, effectiveSessionMinutes / Math.max(5, avgConsultMinutes));
155
156         // Use the smaller of the doctor's configured max and the calculated capacity
157         long maxQueueSize = doctor.getMaxQueueSize() > 0

```

```

158         ? Math.min(doctor.getMaxQueueSize(), calculatedMaxCapacity)
159         : calculatedMaxCapacity;
160
161     int waitingCount = queueRepository.countWaitingByDoctor(doctorId);
162     if (waitingCount >= maxQueueSize) {
163         throw new IllegalStateException("The queue for " + doctor.getName()
164             + " has reached today's session limit (" + maxQueueSize
165             + " patients based on " + avgConsultMinutes + "-min avg consultation). "
166             + "Please choose another doctor or try again later.");
167     }
168
169     String queueNumber = generateQueueNumber(doctorId);
170     long waitingTime = waitingTimeService.getPredictedWaitTime(doctorId);
171
172     Queue queue = new Queue(queueNumber, patientId, doctorId, departmentId, priority, Queue.STATUS_WAITING);
173     queue.setPosition(waitingCount + 1);
174     queue.setEstimatedWaitingTime(waitingTime);
175     queue.setSource(source != null ? source : Queue.SOURCE_ONLINE);
176     queue.setEmergency(emergency);
177     queue.setEmergencyConfirmed(false);
178     queue.setCreatedAt(LocalDateTime.now());
179     queue.setCheckedInAt(LocalDateTime.now());
180
181     queueRepository.insert(queue);
182
183     // Link symptoms to queue (many-to-many)
184     if (symptomIds != null && !symptomIds.isEmpty()) {
185         symptomRepository.linkQueueSymptoms(queue.getQueueId(), symptomIds);
186     }
187
188     notificationService.notify(patientId,
189         "Your queue number " + queueNumber + " has been created for " + doctor.getName()
190         + ". Estimated wait: " + waitingTime + " minutes.");
191
192     if (emergency) {
193         notificationService.notify(patientId,
194             "Your case has been flagged as an emergency and is awaiting staff confirmation for
priority.");
195     }
196     return queue;
197 }
198
199 /**
200  * Rule 9: checks in a patient who has an appointment today with appointment
201  * priority.
202  */
203 public Queue checkInAppointment(String patientId) {
204     Queue activeQueue = queueRepository.findActiveQueueByPatientId(patientId);
205     if (queueRule.hasActiveQueue(activeQueue)) {
206         throw new IllegalStateException("Rule 1: you already have an active queue. Cancel it first.");
207     }
208
209     Appointment appointment = appointmentRepository.findTodaysAppointment(patientId);
210     if (!queueRule.hasTodaysAppointment(appointment)) {
211         throw new IllegalStateException("Rule 9: no appointment found for today.");
212     }
213
214     Queue queue = createQueue(patientId, appointment.getDoctorId(), appointment.getDepartmentId(),
215         Queue.PRIORITY_APPOINTMENT, false, Queue.SOURCE_ONLINE, List.of());
216     appointmentRepository.updateStatus(appointment.getAppointmentId(), Appointment.STATUS_CHECKED_IN,
217         queue.getQueueId());
218     return queue;
219 }
220
221 public boolean cancelQueue(String patientId) {
222     Queue activeQueue = queueRepository.findActiveQueueByPatientId(patientId);

```

```

223     if (!queueRule.hasActiveQueue(activeQueue)) {
224         throw new IllegalStateException("No active queue to cancel.");
225     }
226     queueRepository.cancel(activeQueue.getQueueId(), "Patient cancelled");
227     notificationService.notify(patientId,
228         "Your queue number " + activeQueue.getQueueNumber() + " has been cancelled.");
229     return true;
230 }
231
232 public boolean expireQueue(long queueId) {
233     Queue queue = queueRepository.findById(queueId);
234     if (queue == null) {
235         return false;
236     }
237     queueRepository.expire(queueId);
238     notificationService.notify(queue.getPatientId(),
239         "Your queue number " + queue.getQueueNumber()
240         + " has expired because you missed your turn. Please register again.");
241     return true;
242 }
243
244 public Queue getActiveQueue(String patientId) {
245     return queueRepository.findActiveQueueByPatientId(patientId);
246 }
247
248 /**
249  * Doctor: call the next patient using priority ordering (Rule 2, 4).
250  * The patient is marked CALLED until the doctor starts the consultation.
251  * Also updates estimated waiting times for remaining patients.
252  */
253 public Queue callNext(String doctorId) {
254     if (currentConsultation(doctorId) != null) {
255         throw new IllegalStateException(
256             "Finish or start the current consultation before calling the next patient.");
257     }
258     List<Queue> waiting = queueRepository.findWaitingQueuesByDoctor(doctorId);
259     if (waiting.isEmpty()) {
260         return null;
261     }
262     Queue next = emergencyRule.reorderByPriority(waiting).get(0);
263     queueRepository.callNext(next.getQueueId());
264     notificationService.notify(next.getPatientId(),
265         "It's your turn! Queue number " + next.getQueueNumber()
266         + " is now being served. Please proceed to the doctor.");
267
268     // Update estimated waiting times for remaining patients
269     updateRemainingWaitTimes(doctorId);
270
271     return next;
272 }
273
274 /**
275  * Updates estimated waiting times for all remaining WAITING patients
276  * for a given doctor, based on current queue position and historical data.
277  */
278 private void updateRemainingWaitTimes(String doctorId) {
279     try {
280         Doctor doctor = doctorRepository.findById(doctorId);
281         if (doctor == null) return;
282         long avgConsult = waitingTimeService.getDoctorAverageConsultationMinutes(doctor);
283         List<Queue> remaining = queueRepository.findWaitingQueuesByDoctor(doctorId);
284         java.util.Map<Long, Long> waitByQueueId = new java.util.LinkedHashMap<>();
285         for (int i = 0; i < remaining.size(); i++) {
286             Queue q = remaining.get(i);
287             int patientsAhead = i; // patients ahead of this one (0-indexed)
288             long estimatedWait = patientsAhead * avgConsult;

```

```

289         // Add break offset if needed
290         java.time.LocalDateTime now = java.time.LocalDateTime.now();
291         java.time.LocalDateTime completion = now.plusMinutes(estimatedWait);
292         if (now.isBefore(java.time.LocalDateTime.of(13, 0))
293             && completion.isAfter(java.time.LocalDateTime.of(12, 0))) {
294             estimatedWait += 60; // lunch break
295         }
296         waitByQueueId.put(q.getQueueId(), Math.max(0, estimatedWait));
297     }
298     // One batched round trip instead of one UPDATE per waiting patient -
299     // this runs on every call-next/complete action, so it's a hot path.
300     queueRepository.updateEstimatedWaitingTimes(waitByQueueId);
301 } catch (Exception e) {
302     // Non-critical: log but don't fail the call-next operation
303     org.slf4j.LoggerFactory.getLogger(QueueService.class)
304         .warn("Failed to update remaining wait times for doctor {}", doctorId, e);
305 }
306 }
307
308 /**
309  * Doctor: start the consultation for the currently called patient.
310  */
311 public boolean startConsultation(String doctorId) {
312     Queue called = called(doctorId);
313     if (called == null) {
314         return false;
315     }
316     queueRepository.start(called.getQueueId());
317     notificationService.notify(called.getPatientId(),
318         "Consultation has started for queue number " + called.getQueueNumber() + ".");
319     return true;
320 }
321
322 /**
323  * The patient currently called (waiting to start) for a doctor, if any.
324  */
325 public Queue called(String doctorId) {
326     List<Queue> list = queueRepository.findCalledByDoctor(doctorId);
327     return list.isEmpty() ? null : list.get(0);
328 }
329
330 /**
331  * The active consultation for a doctor: a called patient not yet started,
332  * or the patient currently being served.
333  */
334 public Queue currentConsultation(String doctorId) {
335     Queue called = called(doctorId);
336     if (called != null) {
337         return called;
338     }
339     return serving(doctorId);
340 }
341
342 public boolean completeCurrent(String doctorId) {
343     Queue serving = serving(doctorId);
344     if (serving == null) {
345         return false;
346     }
347     queueRepository.complete(serving.getQueueId());
348     notificationService.notify(serving.getPatientId(),
349         "Your consultation for queue number " + serving.getQueueNumber() + " has been completed. Thank
you!");
350     // Update estimated waiting times for remaining patients after completion
351     updateRemainingWaitTimes(doctorId);
352     return true;
353 }

```

```

354
355     public boolean pauseCurrent(String doctorId) {
356         Queue serving = serving(doctorId);
357         if (serving == null) {
358             return false;
359         }
360         queueRepository.pause(serving.getQueueId());
361         return true;
362     }
363
364     public boolean resumeCurrent(String doctorId) {
365         Queue serving = serving(doctorId);
366         if (serving == null) {
367             return false;
368         }
369         queueRepository.resume(serving.getQueueId());
370         return true;
371     }
372
373     public Queue serving(String doctorId) {
374         List<Queue> list = queueRepository.findServingByDoctor(doctorId);
375         return list.isEmpty() ? null : list.get(0);
376     }
377
378     public List<Queue> waitingForDoctor(String doctorId) {
379         return emergencyRule.reorderByPriority(queueRepository.findWaitingQueuesByDoctor(doctorId));
380     }
381
382     public void confirmEmergency(long queueId) {
383         queueRepository.confirmEmergency(queueId);
384         Queue q = queueRepository.findById(queueId);
385         if (q != null) {
386             notificationService.notify(q.getPatientId(),
387                 "Your emergency has been confirmed. You have been moved to the front of the queue.");
388         }
389     }
390
391     public void reassignDoctor(long queueId, String doctorId) {
392         queueRepository.reassignDoctor(queueId, doctorId);
393     }
394
395     private String generateQueueNumber(String doctorId) {
396         String doctorCode = doctorRepository.findById(doctorId).getDoctorCode();
397         String prefix = QueueNumberGenerator.buildPrefix(doctorCode);
398         String latest = queueRepository.findLatestQueueNumber(prefix);
399         int lastSequence = QueueNumberGenerator.extractLastSequence(latest);
400         return QueueNumberGenerator.generateQueueNumber(doctorCode, lastSequence);
401     }
402 }

```

src/main/java/com/hospitalqueue/rule/EmergencyRule.java

66 lines · Priority ordering rule (shared with Member 2)

```

1  package com.hospitalqueue.rule;
2
3  import com.hospitalqueue.model.Queue;
4  import org.springframework.stereotype.Component;
5
6  import java.time.Duration;
7  import java.time.LocalDateTime;
8  import java.util.List;
9
10 /**

```

```

11  * Emergency screening rules (Member 2 - Smart Queue Engine).
12  */
13  @Component
14  public class EmergencyRule {
15
16      /**
17       * Flags a queue as an emergency (pending staff confirmation).
18       */
19      public boolean requiresStaffConfirmation(Queue queue) {
20          return queue != null && queue.isEmergency() && !queue.isEmergencyConfirmed();
21      }
22
23      /**
24       * An emergency is fully active once confirmed by staff.
25       */
26      public boolean isActiveEmergency(Queue queue) {
27          return queue != null && queue.isEmergency() && queue.isEmergencyConfirmed();
28      }
29
30      /**
31       * Reorders a waiting list so that confirmed emergencies are served first.
32       * Order: confirmed emergency -> appointment -> normal, then by position.
33       */
34      public List<Queue> reorderByPriority(List<Queue> waitingQueues) {
35          waitingQueues.sort((q1, q2) -> {
36              int p1 = priorityRank(q1);
37              int p2 = priorityRank(q2);
38              if (p1 != p2) {
39                  return Integer.compare(p1, p2);
40              }
41              return Integer.compare(q1.getPosition(), q2.getPosition());
42          });
43          return waitingQueues;
44      }
45
46      /**
47       * Detects missed queues (expiry rule): a waiting queue older than the
48       * configured expiry minutes is considered missed.
49       */
50      public boolean isMissed(LocalDate createdAt, long expiryMinutes) {
51          if (createdAt == null) {
52              return false;
53          }
54          return Duration.between(createdAt, LocalDateTime.now()).toMinutes() >= expiryMinutes;
55      }
56
57      private int priorityRank(Queue queue) {
58          if (Queue.PRIORITY_EMERGENCY.equals(queue.getPriority())) {
59              return 1;
60          }
61          if (Queue.PRIORITY_APPOINTMENT.equals(queue.getPriority())) {
62              return 2;
63          }
64          return 3;
65      }
66  }

```

```

1  import { createFileRoute } from "@tanstack/react-router";
2  import { useEffect, useState } from "react";
3  import { Activity, CheckCircle2, Pause, Play, RotateCcw, Sparkles, Volume2 } from "lucide-react";
4  import {

```

```

5   doctorCallNext,
6   doctorCompleteConsultation,
7   doctorPauseConsultation,
8   doctorResumeConsultation,
9   doctorStartConsultation,
10  getDoctorDashboard,
11  getDoctorProfile,
12 } from "@services/api";
13 import { DoctorLayout } from "@components/portal/shells";
14 import {
15   DataTable,
16   Panel,
17   QueueTokenBadge,
18   StatusBadge,
19   type Column,
20 } from "@components/portal/ui-kit";
21 import { useAuth } from "@lib/auth";
22 import { SkeletonCard, SkeletonTable, SkeletonPanel } from "@components/ui/loading";
23 import type { DoctorDashboard, QueueEntry } from "@types";
24
25 export const Route = createFileRoute("/doctor/queue")({
26   head: () => ({
27     meta: [
28       { title: "Consultation Queue – Doctor Portal" },
29       { name: "description", content: "Interactive consultation room and live queue caller." },
30       { property: "og:title", content: "Consultation Queue – Doctor Portal" },
31       { property: "og:description", content: "Interactive consultation room." },
32     ],
33   }),
34   component: DoctorQueueRoomPage,
35 });
36
37 function DoctorQueueRoomPage() {
38   const { session } = useAuth();
39   const [data, setData] = useState<DoctorDashboard | null>(null);
40   const [loading, setLoading] = useState(true);
41   const [actionLoading, setActionLoading] = useState(false);
42   const [message, setMessage] = useState<string | null>(null);
43   const [doctorId, setDoctorId] = useState<string | number>(
44     session?.doctorId ?? session?.userId ?? "D001",
45   );
46
47   // Sync doctorId from session
48   useEffect(() => {
49     const currentDocId = session?.doctorId ?? session?.userId;
50     if (currentDocId && currentDocId !== doctorId) {
51       setDoctorId(currentDocId);
52     } else if (!session?.doctorId && session?.name && session?.role === "DOCTOR") {
53       getDoctorProfile(session.name)
54         .then((profile) => {
55           if (profile?.id) {
56             setDoctorId(profile.id);
57           }
58         })
59         .catch(console.error);
60     }
61   }, [session, doctorId]);
62
63   async function loadData() {
64     try {
65       const res = await getDoctorDashboard(doctorId);
66       setData(res);
67     } catch (err) {
68       console.error(err);
69     } finally {
70       setLoading(false);

```

```

71     }
72 }
73
74 useEffect(() => {
75     loadData();
76     const timer = setInterval(loadData, 8000);
77     return () => clearInterval(timer);
78 }, [doctorId]);
79
80 async function handleCallNext() {
81     setActionLoading(true);
82     try {
83         const res = await doctorCallNext(doctorId);
84         setMessage(res.message);
85         await loadData();
86         setTimeout(() => setMessage(null), 4000);
87     } catch (err: any) {
88         setMessage(err?.message ?? "Failed to call next patient");
89         setTimeout(() => setMessage(null), 4000);
90     } finally {
91         setActionLoading(false);
92     }
93 }
94
95 async function handleStart() {
96     setActionLoading(true);
97     try {
98         const res = await doctorStartConsultation(doctorId);
99         setMessage(res.message);
100        await loadData();
101        setTimeout(() => setMessage(null), 4000);
102    } catch (err: any) {
103        setMessage(err?.message ?? "Failed to start consultation");
104        setTimeout(() => setMessage(null), 4000);
105    } finally {
106        setActionLoading(false);
107    }
108 }
109
110 async function handleComplete() {
111     setActionLoading(true);
112     try {
113         const res = await doctorCompleteConsultation(doctorId);
114         setMessage(res.message);
115         await loadData();
116         setTimeout(() => setMessage(null), 4000);
117     } catch (err: any) {
118         setMessage(err?.message ?? "Failed to complete consultation");
119         setTimeout(() => setMessage(null), 4000);
120     } finally {
121         setActionLoading(false);
122     }
123 }
124
125 async function handlePause() {
126     setActionLoading(true);
127     try {
128         const res = await doctorPauseConsultation(doctorId);
129         setMessage(res.message);
130         await loadData();
131         setTimeout(() => setMessage(null), 4000);
132     } catch (err: any) {
133         setMessage(err?.message ?? "Failed to pause consultation");
134         setTimeout(() => setMessage(null), 4000);
135     } finally {
136         setActionLoading(false);

```

```

137     }
138   }
139
140   async function handleResume() {
141     setActionLoading(true);
142     try {
143       const res = await doctorResumeConsultation(doctorId);
144       setMessage(res.message);
145       await loadData();
146       setTimeout(() => setMessage(null), 4000);
147     } catch (err: any) {
148       setMessage(err?.message ?? "Failed to resume consultation");
149       setTimeout(() => setMessage(null), 4000);
150     } finally {
151       setActionLoading(false);
152     }
153   }
154
155   const columns: Column<QueueEntry>[] = [
156     {
157       header: "Queue No.",
158       cell: (r) => <span className="font-bold text-primary">{r.queueNumber}</span>,
159     },
160     { header: "Patient Name", cell: (r) => <span className="font-medium">{r.patientName}</span> },
161     {
162       header: "Type",
163       cell: (r) => (
164         <div className="flex items-center gap-1.5">
165           <StatusBadge status={r.type ?? "NORMAL"} />
166           {r.emergency && (
167             <span className="rounded bg-danger/20 px-1.5 py-0.5 text-[10px] font-bold text-danger">
168               EMERGENCY
169             </span>
170           )}
171         </div>
172       ),
173     },
174     { header: "Line Position", cell: (r) => `#${r.position}` },
175     { header: "Estimated Wait", cell: (r) => `${r.estimatedWaitingMinutes ?? 15} min` },
176     { header: "Status", cell: (r) => <StatusBadge status={r.status} /> },
177   ];
178
179   const activePatient = data?.serving ?? data?.called;
180   const isServing = !!data?.serving;
181   const isCalled = !!data?.called && !data?.serving;
182
183   if (loading && !data) {
184     return (
185       <DoctorLayout title="Consultation Queue">
186         <div className="space-y-6">
187           <SkeletonCard className="rounded-2xl p-6" />
188           <SkeletonPanel title />
189         </div>
190       </DoctorLayout>
191     );
192   }
193
194   return (
195     <DoctorLayout title="Consultation Queue">
196       <div className="space-y-6">
197         {/* Active Consultation Console */}
198         <div className="rounded-2xl border border-primary/30 bg-card p-4 shadow-sm sm:p-6">
199           <div className="flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-between sm:border-b
sm:border-border sm:pb-4">
200             <div className="flex items-center gap-3">
201               <div className="flex size-9 items-center justify-center rounded-xl bg-primary text-primary-

```

```

foreground shadow-sm sm:size-10">
202         <Activity className="size-5 sm:size-6" />
203     </div>
204     <div>
205         <h2 className="text-lg font-bold text-foreground sm:text-xl">
206             Consultation Console
207         </h2>
208         <p className="text-xs text-muted-foreground">
209             Call, serve, pause, and complete patient queues
210         </p>
211     </div>
212 </div>
213 <button
214     onClick={handleCallNext}
215     disabled={actionLoading || isServing || isCalled}
216     className="flex items-center justify-center gap-2 rounded-xl bg-primary px-4 py-2.5 text-sm font-
bold text-primary-foreground shadow hover:opacity-90 disabled:opacity-50 sm:px-5"
217     >
218         <Volume2 className="size-4" />
219         Call Next Patient
220     </button>
221 </div>
222
223 {message && (
224     <div className="mt-4 flex items-center gap-2 rounded-lg bg-primary/10 px-4 py-2 text-sm font-medium
text-primary">
225         <Sparkles className="size-4 shrink-0" />
226         {message}
227     </div>
228 )}
229
230 <div className="mt-6">
231     {activePatient ? (
232         <div className="flex flex-col gap-4 rounded-xl border border-border bg-accent/30 p-4 sm:flex-row
sm:items-center sm:justify-between sm:p-6">
233         <div className="flex flex-col gap-3 sm:flex-row sm:items-center">
234             <QueueTokenBadge
235                 token={activePatient.queueNumber}
236                 station={
237                     isServing ? "In Consultation" : "Called · Awaiting Arrival"
238                 }
239                 urgency={activePatient.emergency ? "critical" : "general"}
240             />
241             <div>
242                 <span className="text-lg font-bold text-foreground sm:text-xl">
243                     {activePatient.patientName}
244                 </span>
245                 <div className="mt-2 flex flex-wrap items-center gap-2 text-xs text-muted-foreground">
246                     <StatusBadge status={activePatient.type ?? "NORMAL"} />
247                     {activePatient.emergency && (
248                         <span className="rounded bg-danger/20 px-2 py-0.5 font-bold text-danger">
249                             EMERGENCY
250                         </span>
251                     )}
252                     <span>Waiting: {activePatient.estimatedWaitingMinutes ?? 15} min</span>
253                 </div>
254             </div>
255         </div>
256
257         <div className="flex flex-wrap items-center gap-2 sm:gap-3">
258             {isCalled && (
259                 <button
260                     onClick={handleStart}
261                     disabled={actionLoading}
262                     className="flex items-center gap-2 rounded-xl bg-success px-4 py-2 text-xs font-bold text-
success-foreground hover:opacity-90 shadow-sm sm:px-5 sm:py-2.5 sm:text-sm"

```

```

263         >
264         <Play className="size-4 fill-current" />
265         Start Consultation
266     </button>
267 }
268
269 {isServing && (
270     <>
271     <button
272         onClick={handleComplete}
273         disabled={actionLoading}
274         className="flex items-center gap-2 rounded-xl bg-primary px-4 py-2 text-xs font-bold
text-primary-foreground hover:opacity-90 shadow-sm sm:px-5 sm:py-2.5 sm:text-sm"
275     >
276         <CheckCircle2 className="size-4" />
277         Complete
278     </button>
279     <button
280         onClick={handlePause}
281         disabled={actionLoading}
282         className="flex items-center gap-2 rounded-xl bg-secondary px-3 py-2 text-xs font-
semibold text-secondary-foreground hover:bg-secondary/80 sm:px-4 sm:py-2.5 sm:text-sm"
283     >
284         <Pause className="size-4" />
285         Pause
286     </button>
287 </>
288 }
289
290 {activePatient.status === "PAUSED" && (
291     <button
292         onClick={handleResume}
293         disabled={actionLoading}
294         className="flex items-center gap-2 rounded-xl bg-primary px-3 py-2 text-xs font-semibold
text-primary-foreground hover:opacity-90 sm:px-4 sm:py-2.5 sm:text-sm"
295     >
296         <RotateCcw className="size-4" />
297         Resume
298     </button>
299 )}
300 </div>
301 </div>
302 ) : (
303     <div className="py-8 text-center text-muted-foreground text-sm">
304         No patient currently called. Click <strong>"Call Next Patient"</strong> to call the
305         next patient in line.
306     </div>
307 )}
308 </div>
309 </div>
310
311 {/* Full Waiting Queue */}
312 <Panel title={`Waiting Queue (${data?.waiting.length ?? 0} Patients)`}>
313     {loading ? (
314         <SkeletonTable rows={5} columns={6} />
315     ) : (
316         <DataTable
317             rows={data?.waiting ?? []}
318             columns={columns}
319             loading={loading}
320             emptyMessage="No patients currently waiting in queue."
321             pageSize={10}
322         />
323     )}
324 </Panel>
325 </div>

```

```
326     </DoctorLayout>
327   );
328 }
```

frontend/src/routes/staff.emergency.tsx

226 lines · Staff emergency-cases page

```
1 import { createFileRoute } from "@tanstack/react-router";
2 import { useEffect, useState } from "react";
3 import { AlertTriangle, Info, ShieldAlert, Timer } from "lucide-react";
4 import { confirmEmergency, getEmergencyCases, rejectEmergency } from "@services/api";
5 import { StaffLayout } from "@components/portal/shells";
6 import { DataTable, Panel, StatusBadge, TabNav, type Column } from "@components/portal/ui-kit";
7 import { SkeletonTable } from "@components/ui/loading";
8 import type { EmergencyCase } from "@types";
9
10 export const Route = createFileRoute("/staff/emergency")({
11   head: () => ({
12     meta: [
13       { title: "Emergency Cases – Staff Portal" },
14       { name: "description", content: "Confirm or reject emergency queue requests from patients." },
15       { property: "og:title", content: "Emergency Cases – Staff Portal" },
16       { property: "og:description", content: "Confirm or reject emergency queue requests." },
17     ],
18   }),
19   component: EmergencyPage,
20 });
21
22 function EmergencyPage() {
23   const [cases, setCases] = useState<EmergencyCase[]>([]);
24   const [loading, setLoading] = useState(true);
25   const [tab, setTab] = useState("pending");
26
27   useEffect(() => {
28     getEmergencyCases()
29       .then((data) => {
30         setCases(data);
31         setLoading(false);
32       })
33       .catch((err) => {
34         console.error(err);
35         setLoading(false);
36       });
37   }, []);
38
39   async function handleConfirm(id: number) {
40     const updated = await confirmEmergency(id);
41     setCases((prev) => prev.map((c) => (c.id === id ? updated : c)));
42   }
43
44   async function handleReject(id: number) {
45     await rejectEmergency(id);
46     setCases((prev) => prev.filter((c) => c.id !== id));
47   }
48
49   const pending = cases.filter((c) => !c.resolved);
50   const resolved = cases.filter((c) => c.resolved);
51   const critical = pending.filter((c) => c.priority === "HIGH");
52   const fastTrack = pending.filter((c) => c.priority !== "HIGH");
53
54   const pendingColumns: Column<EmergencyCase>[] = [
55     {
56       header: "Patient ID",
57       cell: (r) => <span className="font-medium text-primary">{r.patientCode}</span>,
```

```

58   },
59   { header: "Patient Name", cell: (r) => r.patientName },
60   { header: "Symptoms", cell: (r) => r.symptoms },
61   { header: "Requested Time", cell: (r) => r.requestedTime },
62   { header: "Priority", cell: (r) => <StatusBadge status={r.priority} /> },
63   {
64     header: "Action",
65     cell: (r) => (
66       <div className="flex gap-2">
67         <button
68           onClick={() => handleConfirm(r.id)}
69           className="rounded-md bg-success-soft px-3 py-1 text-xs font-medium text-success"
70         >
71           Confirm
72         </button>
73         <button
74           onClick={() => handleReject(r.id)}
75           className="rounded-md bg-danger-soft px-3 py-1 text-xs font-medium text-danger"
76         >
77           Reject
78         </button>
79       </div>
80     ),
81   },
82 ];
83
84 const resolvedColumns: Column<EmergencyCase>[] = [
85   { header: "Patient ID", cell: (r) => r.patientCode },
86   { header: "Patient Name", cell: (r) => r.patientName },
87   { header: "Confirmed Time", cell: (r) => r.confirmedTime ?? "-" },
88   { header: "Handled By", cell: (r) => r.handledBy ?? "-" },
89   { header: "Priority", cell: (r) => <StatusBadge status={r.priority} /> },
90 ];
91
92 return (
93   <StaffLayout title="Emergency Cases">
94     <div className="space-y-6">
95       {/* Critical metrics & rapid protocol banner */}
96       <div className="grid gap-4 xl:grid-cols-12">
97         <div className="relative overflow-hidden rounded-xl border border-danger/30 bg-danger-soft p-5 xl:col-
span-4">
98           <div className="flex items-start justify-between">
99             <div className="flex items-center gap-2.5">
100               <ShieldAlert className="size-6 animate-pulse text-danger" />
101               <div>
102                 <span className="block text-[10px] font-bold uppercase tracking-wider text-danger">
103                   Red Triage Protocol
104                 </span>
105                 <h2 className="text-base font-bold text-foreground">High Priority Cases</h2>
106               </div>
107             </div>
108             {critical.length > 0 ? (
109               <span className="animate-bounce rounded-full bg-danger px-2 py-0.5 text-[10px] font-bold text-
danger-foreground">
110                 ACT NOW
111               </span>
112             ) : null}
113           </div>
114           <div className="mt-4 grid grid-cols-2 gap-3">
115             <div className="rounded-lg border border-border bg-card p-3 shadow-sm">
116               <span className="text-2xl font-extrabold text-danger">{critical.length}</span>
117               <p className="text-xs font-semibold text-foreground">Critical Cases</p>
118               <p className="text-[11px] font-medium text-danger">Immediate attention</p>
119             </div>
120             <div className="rounded-lg border border-border bg-card p-3 shadow-sm">
121               <span className="text-2xl font-extrabold text-warning">{fastTrack.length}</span>

```

```

122         <p className="text-xs font-semibold text-foreground">Fast-Track Pending</p>
123         <p className="text-[11px] font-medium text-warning">Priority queue</p>
124     </div>
125 </div>
126 </div>
127
128 <div className="rounded-xl border border-border bg-card p-5 xl:col-span-8">
129     <div className="flex flex-wrap items-center justify-between gap-2 pb-3">
130         <div className="flex items-center gap-2">
131             <AlertTriangle className="size-5 text-primary" />
132             <span className="text-base font-bold text-foreground">Rapid Triage Directive</span>
133         </div>
134         <div className="flex items-center gap-1.5 text-xs text-muted-foreground">
135             <Timer className="size-3.5" />
136             Target Door-to-Doctor: <strong className="text-foreground">&lt; 10 mins</strong>
137         </div>
138     </div>
139     <div className="grid gap-3 border-t border-border pt-3 sm:grid-cols-3">
140         <div className="flex items-start gap-2 rounded-lg bg-muted/50 p-3">
141             <AlertTriangle className="mt-0.5 size-4 shrink-0 text-danger" />
142             <div>
143                 <p className="text-xs font-bold text-foreground">Acute chest pain / cardiac</p>
144                 <p className="text-[11px] text-muted-foreground">Route straight to on-call doctor.</p>
145             </div>
146         </div>
147         <div className="flex items-start gap-2 rounded-lg bg-muted/50 p-3">
148             <AlertTriangle className="mt-0.5 size-4 shrink-0 text-warning" />
149             <div>
150                 <p className="text-xs font-bold text-foreground">Breathing difficulty</p>
151                 <p className="text-[11px] text-muted-foreground">Fast-track ahead of the queue.</p>
152             </div>
153         </div>
154         <div className="flex items-start gap-2 rounded-lg bg-muted/50 p-3">
155             <Info className="mt-0.5 size-4 shrink-0 text-info" />
156             <div>
157                 <p className="text-xs font-bold text-foreground">Everything else</p>
158                 <p className="text-[11px] text-muted-foreground">Confirm case, assign standard priority.</p>
159             </div>
160         </div>
161     </div>
162 </div>
163 </div>
164
165 <Panel>
166     <TabNav
167         value={tab}
168         onChange={setTab}
169         tabs={[
170             { value: "pending", label: `Pending (${pending.length})` },
171             { value: "resolved", label: "Resolved" },
172         ]}
173     />
174     {tab === "pending" ? (
175         loading ? (
176             <SkeletonTable rows={5} columns={6} />
177         ) : (
178             <DataTable
179                 rows={pending}
180                 columns={pendingColumns}
181                 loading={loading}
182                 emptyMessage="No pending emergency cases."
183             />
184         )
185     ) : loading ? (
186         <SkeletonTable rows={5} columns={5} />
187     ) : (

```

```

188         <DataTable
189             rows={resolved}
190             columns={resolvedColumns}
191             loading={loading}
192             emptyMessage="No confirmed emergency yet."
193         />
194     )}
195 </Panel>
196
197 <div className="grid gap-6 lg:grid-cols-2">
198     <Panel>
199         <h3 className="mb-3 flex items-center gap-2 text-sm font-semibold text-foreground">
200             <Info className="size-4 text-info" /> What to do?
201         </h3>
202         <ol className="list-decimal space-y-1.5 pl-5 text-sm text-muted-foreground">
203             <li>Check the patient's symptoms.</li>
204             <li>Confirm if it is a real emergency.</li>
205             <li>If yes, set priority to Emergency.</li>
206             <li>If no, continue as normal queue.</li>
207         </ol>
208     </Panel>
209     <Panel title="Confirmed Emergency">
210         {loading ? (
211             <SkeletonTable rows={3} columns={5} />
212         ) : (
213             <DataTable
214                 rows={resolved}
215                 columns={resolvedColumns}
216                 loading={loading}
217                 emptyMessage="No confirmed emergency yet."
218                 pageSize={5}
219             />
220         )}
221     </Panel>
222 </div>
223 </div>
224 </StaffLayout>
225 );
226 }

```